Table of Contents

Table of Contents **1**
CaLLP AWS Infrastructure and Deployment Guide **2**
1. Prerequisites 2
2. Repository layout 2
3. Development environment 3
4. Environment configuration and parameters 4
5. Deploy an application version 4
6. HTTPS automation and verification 5
7. Recreate an environment **6**
7.1 Verify existing resources 6
7.2 Create the environment 7
7.3 Verify environment readiness 9
7.4 Verify SSM configuration and HTTPS behavior 10
7.5 Clean up a temporary validation environment (destructive) 12
8. Troubleshooting and rollback 15
9. Repository and security rules 15

CaLLP AWS Infrastructure and Deployment Guide

This guide documents the CaLLP development environment hosted on AWS Elastic Beanstalk (EB). It covers configuration preservation, application deployment, HTTPS recovery, and safe environment recreation. The deployed application is currently the Elastic Beanstalk Python sample application; Django integration is not yet verified.

1. Prerequisites

- AWS access to the **us-west-2** region and the CaLLP Elastic Beanstalk application.
- Access to AWS CloudShell or the AWS CLI, and permission to view Elastic Beanstalk, EC2, RDS, IAM, and SSM resources.
- A local clone of `callp-app` for building application bundles.
- Existing external dependencies: VPC/subnets, RDS database, persistent security groups, SSM parameters, IAM roles, and Porkbun DNS.

*Do **NOT** use the AWS root account for routine deployment or administration.*

2. Repository layout

Textproto

```
callp-app/
├── .ebextensions/
│   └── 10_web_security.config
├── .platform/
│   ├── hooks/
│   ├── confighooks/
│   ├── nginx/
│   ├── scripts/
│   ├── systemd/
│   └── templates/
├── infrastructure/
│   └── elasticbeanstalk/
│       └── callp-dev-ssm-baseline_v1.cfg.yml
```

```
└── docs/
    └── deployment/
        └── aws-deployment-guide.pdf
```

- `.ebextensions/` manages environment-specific ingress resources through Elastic Beanstalk CloudFormation.
- `.platform/` installs and maintains Nginx HTTPS, Certbot, and systemd services.
- `infrastructure/elasticbeanstalk/` stores a reviewed AWS-native environment configuration for recovery; it is **not** applied automatically during ordinary deployments.

3. Development environment

Setting	Value
AWS region	`us-west-2`
EB application	`Career and Lifelong Learning Pathway`
Environment	`CaLLP-Dev-Recovery-env`
Platform	Python 3.14 / Amazon Linux 2023 (baseline: 4.13.10)
Instance type	`t3.micro`
Environment type	Single instance
Public domain	`callp.dev`
Database	Independent RDS MySQL instance `callp-dev-db`
Current tested sample-app version	`callp-https-test-20261015-v3`

Resource IDs and platform versions may change. Verify current AWS state before provisioning or modifying an environment.

4. Environment configuration and parameters

The AWS saved configuration is named **`callp-dev-ssm-baseline_v1`**. A reviewed copy is stored in `infrastructure/elasticbeanstalk/`. It preserves environment settings and six SSM references, but **does not create the referenced parameters or their values**.

EB property	Source
`DJANGO_SECRET_KEY`	`/callp/dev/django/secret_key`
`RDS_DB_NAME`	`/callp/dev/rds/db_name`
`RDS_HOSTNAME`	`/callp/dev/rds/host`
`RDS_PORT`	`/callp/dev/rds/port`
`RDS_USERNAME`	`/callp/dev/rds/username`
`RDS_PASSWORD`	`/callp/dev/rds/password`

All six are configured as **Parameter Store** sources in Elastic Beanstalk. `PYTHONPATH` appears in the expanded environment settings but is omitted from the AWS-native saved file; do not hardcode an instance-specific virtual-environment path without testing.

The EC2 instance profile `aws-elasticbeanstalk-ec2-role` requires permission to retrieve the referenced SSM parameters. The password uses an SSM `SecureString`. Keep secret **values** out of Git, logs, and exported documentation.

5. Deploy an application version

1. Build a ZIP with `.ebextensions/`, `.platform/`, and application files at the archive root; exclude `.DS_Store`, `.env`, private keys, and local artifacts.
2. In **Elastic Beanstalk** → **Application versions**, upload/register a new, uniquely named application version.
3. Deploy the version to `CaLLP-Dev-Recovery-env` through the AWS Console.
4. Wait for environment status **`Ready`** and health **`Green/Ok`**.

5. **Verify HTTPS from a terminal:**

```shell
curl -I --connect-timeout 5 --max-time 15 https://callp.dev/
```

- Expected: HTTP `200 OK`. Keep the previous known-good application version available for rollback when practical. An application deployment is **not** an environment rebuild.

6. HTTPS automation and verification

Deployment event	Hook
Application deployment	`.platform/hooks/postdeploy/20_install_https_services.sh`
Configuration-only deployment	`.platform/confighooks/postdeploy/20_restore_https.sh`

The existing bootstrap service checks DNS and certificates, configures Nginx, and enables renewal after successful HTTPS activation. A systemd timer retries bootstrap when needed. The configuration hook starts the existing service immediately after configuration deployment, avoiding reliance on the next timer interval.

The configuration hook executed successfully in two October 15, 2026 tests (adding and removing a temporary environment property); HTTPS subsequently returned `200 OK`. **Zero downtime during configuration updates has not been established.**

On the Elastic Beanstalk EC2 instance, verify the bootstrap service, retry timer, and listeners:

```shell

sudo systemctl status callp-https-bootstrap.service --no-pager
sudo systemctl status callp-https-bootstrap.timer --no-pager
sudo ss -ltnp | grep -E ':(80|443)\b'
sudo grep -nE \
  'confighooks/postdeploy|20_restore_https|config-deploy' \
  /var/log/eb-engine.log | tail -n 30
```

7. Recreate an environment

Use the AWS Elastic Beanstalk saved configuration `callp-dev-ssm-baseline_v1` to recreate the CaLLP development environment. The saved configuration restores environment settings but does not recreate external resources such as RDS, SSM parameters, IAM roles, or DNS records.

7.1 Verify existing resources

Before creating an environment, confirm the RDS database, persistent security group, saved configuration, and application version are available.

0. Before starting — Set environment variables.

AWS CloudShell:

```shell
REGION="us-west-2"
APP="Career and Lifelong Learning Pathway"
ENV_NAME="NEW_ENVIRONMENT_NAME"
```

- Replace NEW_ENVIRONMENT_NAME with the unique name of the environment being created. These variables are reused throughout **Section 7**. Keep the same CloudShell session open, or initialize them again if reconnecting.

1. Verify the RDS database.

AWS CloudShell:

```shell
aws rds describe-db-instances \
  --region us-west-2 \
  --db-instance-identifier callp-dev-db \
  --query
'DBInstances[0].{Status:DBInstanceStatus,DeletionProtection:DeletionProtec
tion,PubliclyAccessible:PubliclyAccessible}' \
  --output table --no-cli-pager
```

- Expected: `available`, deletion protection enabled, and public accessibility disabled.

2. Verify the saved configuration and application version.

AWS CloudShell:

```
Shell
aws elasticbeanstalk describe-configuration-settings \
  --region us-west-2 \
  --application-name "Career and Lifelong Learning Pathway" \
  --template-name "callp-dev-ssm-baseline_v1" \
  --query 'ConfigurationSettings[0].TemplateName' \
  --output text --no-cli-pager

aws elasticbeanstalk describe-application-versions \
  --region us-west-2 \
  --application-name "Career and Lifelong Learning Pathway" \
  --version-labels "callp-https-test-20261015-v3" \
  --query 'ApplicationVersions[].{Version:VersionLabel,Status:Status}' \
  --output table --no-cli-pager
```

- Expected: `callp-dev-ssm-baseline_v1` exists and the application version reports `PROCESSED`.
- Also confirm that the VPC, subnets, IAM instance profile, persistent application security group, and six SSM parameters still exist.

7.2 Create the environment

The AWS Console supports launching environments from saved configurations, but CaLLP testing encountered an unresolved `AppSource` validation error when selecting an existing application version**. The AWS CLI method below (step 4) has been successfully tested and is recommended for complete environment recreation. The Console procedure is retained for reference.**

3. ~~Create the environment through the AWS Console.~~

1. ~~Open **Elastic Beanstalk → Applications → Career and Lifelong Learning Pathway → Saved configurations**.~~
2. ~~Select `callp-dev-ssm-baseline_v1`.~~
3. ~~Choose **Actions → Launch environment**.~~
4. ~~Enter a unique environment name.~~
5. ~~Select the appropriate Python platform.~~
6. ~~Under **Application code**, select **Existing version**.~~

7. Under **Application version**, select `callp-https-test-20261015-v3`.
8. Review the inherited configuration, including the persistent security group and SSM references.
9. Select **Create environment**.

Known issue: During October 15, 2026 validation, this Console workflow returned:

```
Configuration validation exception:
    Unknown or duplicate parameter: AppSource
```

CloudTrail confirmed that the Console submitted 74 expanded configuration settings, including the internal `AppSource` parameter, alongside the selected application version. The same 74 settings succeeded without an application version.

This strongly suggests a conflict between the explicitly submitted `AppSource` setting and `versionLabel`, although AWS's internal validation behavior has not been confirmed.

This is a suspected bug: Do NOT modify the v3 bundle, security-group ingress rules, or saved configuration to work around this issue.

4. Use the verified AWS CLI procedure if Console creation fails.

AWS CloudShell:

```shell
aws elasticbeanstalk create-environment \
  --region "$REGION" \
  --application-name "$APP" \
  --environment-name "$ENV_NAME" \
  --template-name "callp-dev-ssm-baseline_v1" \
  --version-label "callp-https-test-20261015-v3" \
  --no-cli-pager
```

- The environment name is taken from `$ENV_NAME`, initialized in Section 7.1. Confirm it identifies the intended new environment before executing this command.
- **Warning:** This command creates billable AWS resources. Do **NOT** reuse the name of an existing environment.
- Unlike the failed Console request, the CLI references the saved configuration directly without explicitly submitting internal CloudFormation parameters.

- **Expected:** AWS returns a new environment ID, the selected application version, and an initial status of `Launching`.

7.3 Verify environment readiness

5. Retrieve the environment ID and hostname.

AWS CloudShell:

```shell
ENV_ID=$(aws elasticbeanstalk describe-environments \
  --region "$REGION" \
  --environment-names "$ENV_NAME" \
  --no-include-deleted \
  --query 'Environments[0].EnvironmentId' \
  --output text)

if [[ ! "$ENV_ID" =~ ^e-[a-z0-9]+$ ]]; then
  echo "ERROR: Invalid environment ID. Stop here and check ENV_NAME."
fi

CNAME=$(aws elasticbeanstalk describe-environments \
  --region "$REGION" \
  --environment-ids "$ENV_ID" \
  --query 'Environments[0].CNAME' \
  --output text)

printf 'Environment: %s\nEnvironment ID: %s\nHostname: %s\n' \
  "$ENV_NAME" "$ENV_ID" "$CNAME"
```

- Confirm the returned identifiers belong to the newly created environment. If any value is `missing` or `None`, stop and investigate before continuing.
- Stop if an error is displayed. Do not continue until the environment ID is valid.

6. Verify environment health and application version.

AWS CloudShell:

```shell
aws elasticbeanstalk describe-environments \
  --region "$REGION" \
  --environment-ids "$ENV_ID" \
  --query
'Environments[0].{Status:Status,Health:Health,Version:VersionLabel}' \
  --output table --no-cli-pager
```

- Expected: `Ready`, `Green`, and `callp-https-test-20261015-v3`.

7. Verify HTTP availability.

AWS CloudShell:

```shell
curl -I --connect-timeout 5 --max-time 15 "http://${CNAME}/"
```

- **Expected:** `HTTP/1.1 200 OK`. Do not change live DNS solely to test a parallel environment.

7.4 Verify SSM configuration and HTTPS behavior

Connect to the **new environment's EC2 instance**, not the original environment.

8. Verify that all six SSM-backed properties are available to the running application.

Validation EC2:

```python
sudo python3 - <<'PY'
from pathlib import Path
import subprocess
expected = [
    "DJANGO_SECRET_KEY",
    "RDS_DB_NAME",
    "RDS_HOSTNAME",
```

```
    "RDS_PORT",
    "RDS_USERNAME",
    "RDS_PASSWORD",
]
pid = int(subprocess.check_output(
    ["systemctl", "show", "web.service", "-p", "MainPID", "--value"],
    text=True
).strip())
if pid <= 0:
    raise SystemExit("ERROR: web.service is not running")
entries = Path("/proc/{}/environ".format(pid)).read_bytes().split(b"\0")
variables = dict(
    entry.split(b"=", 1)
    for entry in entries
    if b"=" in entry
)
for name in expected:
    value = variables.get(name.encode())
    status = "PRESENT" if value else "MISSING OR EMPTY"
    print("{}: {}".format(name, status))
PY
```

- **Expected:** all six variables report PRESENT.
- This verifies that the application process received nonempty values. It does **NOT** independently verify database authentication or compare values against Parameter Store.

9. Verify HTTPS bootstrap and retry services.

Validation EC2:

Shell

```
sudo systemctl status callp-https-bootstrap.service --no-pager
sudo systemctl status callp-https-bootstrap.timer --no-pager

sudo journalctl -u callp-https-bootstrap.service \
  --no-pager -n 30
```

- Do not change the live DNS record or manually request certificates during this test.

When `callp.dev` still points to the original environment, the bootstrap should report: `[callp-https] :`

```
Shell
   ERROR: DNS does not match this instance; retry later
```

- The bootstrap service may show `failed` in this situation. This is expected for a parallel validation environment, and the retry timer should remain active.

7.5 Clean up a temporary validation environment (destructive)

Terminate temporary environments through the AWS Elastic Beanstalk Console. Elastic Beanstalk normally removes its managed EC2 instance, security group, CloudFormation stack, and Elastic IP automatically. Independently managed resources, including the persistent application security group and RDS database, must be preserved.

10. Confirm the temporary environment.

AWS CloudShell:

```
Shell
aws elasticbeanstalk describe-environments \
  --region "$REGION" \
  --environment-ids "$ENV_ID" \
  --query 'Environments[0].{Name:EnvironmentName,Status:Status}' \
  --output table --no-cli-pager
```

- Confirm the returned environment name matches `$ENV_NAME` and is not the working `CaLLP-Dev-Recovery-env`.

11. Check security-group dependencies.

In the EC2 Console, open the temporary environment's running instance and identify its EB-managed security group (`AWSEBSecurityGroup`). Do not select the persistent `callp-dev-app-sg`.

In AWS CloudShell, check for rules referencing the temporary group:

```shell
TEMP_SG_ID="REPLACE_WITH_EB_MANAGED_SG_ID"

if [[ ! "$TEMP_SG_ID" =~ ^sg-[0-9a-f]{8,17}$ ]]; then
  echo "ERROR: Invalid security group ID. Stop here."
else
  aws ec2 describe-security-group-rules \
    --region "$REGION" \
    --query
"SecurityGroupRules[?ReferencedGroupInfo.GroupId=='${TEMP_SG_ID}'].{RuleId
:SecurityGroupRuleId,OwnerGroup:GroupId}" \
    --output table --no-cli-pager
fi
```

- Replace `REPLACE_WITH_EB_MANAGED_SG_ID` with the temporary environment's EB-managed security group ID.
- If rules are returned, inspect each rule's owning security group. Rules belonging to the temporary environment may be removed automatically during termination; rules owned by independently managed infrastructure require investigation.
- Do not manually delete security-group rules unless their ownership and dependencies are understood. Never delete the persistent `callp-dev-app-sg`.

12. Terminate the temporary environment.

1. Open **Elastic Beanstalk → Environments**.
2. Select the temporary environment and confirm its name and ID.
3. Choose **Actions → Terminate environment**.
4. Wait for termination to complete.

Warning: Termination removes the environment's managed infrastructure. Never terminate the working CaLLP environment during validation cleanup.

13. Verify termination.

AWS CloudShell:

```shell
aws elasticbeanstalk describe-environments \
  --region "$REGION" \
  --environment-ids "$ENV_ID" \
  --query 'Environments[0].Status' \
  --output text --no-cli-pager
```

- **Expected:** `Terminated`.
- If termination fails or leaves resources behind, inspect the environment's CloudFormation stack, EC2 instance, security group, and Elastic IP before attempting manual cleanup.

14. Confirm the original environment and RDS remain healthy.

AWS CloudShell:

```shell
aws elasticbeanstalk describe-environments \
  --region "$REGION" \
  --environment-names "CaLLP-Dev-Recovery-env" \
  --query 'Environments[0].{Status:Status,Health:Health}' \
  --output table --no-cli-pager

aws rds describe-db-instances \
  --region "$REGION" \
  --db-instance-identifier callp-dev-db \
  --query
   'DBInstances[0].{Status:DBInstanceStatus,DeletionProtection:Deletion
   Protection}' \
  --output table --no-cli-pager
```

- **Expected:** original environment `Ready / Green`; RDS `available` with deletion protection enabled.

Validation note: On October 15, 2026, complete environment recreation using the saved configuration and v3 application version succeeded through AWS CLI. All six SSM-backed properties were present in the application process, HTTP returned `200 OK`, and the HTTPS DNS safeguard operated correctly. Temporary EC2, CloudFormation, security-group, and Elastic IP cleanup was subsequently verified.

Django-to-RDS authentication and Console-only recreation with v3 remain unverified.

8. Troubleshooting and rollback

- **HTTP works but HTTPS refuses connections:** Check the Nginx `:443` listener, bootstrap service, timer, and `/var/log/eb-engine.log`. Configuration-only deployments must run `.platform/confighooks/postdeploy/`.
- **Certificate issuance delayed:** Check DNS resolution, public IP, and the HTTP-01 challenge path before retrying. Avoid unnecessary certificate requests.
- **SSM resolution fails:** Verify the parameter ARN, existence, source type (**Parameter Store**, not Plain text), instance-role permissions, and any relevant KMS permissions.
- **CloudFormation reports duplicate ingress rules:** Do not add or delete rules blindly. Compare live rules with `.ebextensions/10_web_security.config` and CloudFormation ownership.
- **Environment deletion is blocked:** Check for external security-group references to EB-managed resources. Keep RDS attached to persistent infrastructure instead.
- **Application deployment fails:** Review Elastic Beanstalk Events and logs, then redeploy a known-good application version. Do not rebuild the environment as a first response.

9. Repository and security rules

- Commit reviewed `.ebextensions/`, `.platform/`, and saved `.cfg.yml` configuration.
- Never commit `.env`, secret values, private keys, database dumps, or unsanitized AWS exports.
- Saved configuration contains AWS account/network identifiers and is specific to this environment; review it before publishing or using it in another account.
- Changes to `.ebextensions` can update CloudFormation resources; review infrastructure changes separately from ordinary Django deployments.

- **Future work:** GitHub Actions deployment using OIDC, read-only security-group conflict checks, and automated configuration drift detection. CLI-based environment recreation has been verified; Console-only recreation with v3 remains unresolved.